DEEP LEARNING SEMESTER PROJECT

AI-Based Car Parts Detection and Segmentation Using YOLOv8

Technical Project Report | Academic Submission

Model: YOLOv8n-seg | 23 car-part classes | 25 training epochs

Institution: Superior University, Lahore
Program: BS Computer Science
Prepared by: Rafaqat Ali
Date: 31 October 2026

Public repository: https://github.com/rafaqatali-dev/AI-Car-Parts-Detection-YOLOv8

Deployed frontend: https://ai-car-parts-detection-yolov8-front.vercel.app/

Scope note: This release performs car detection and car-part segmentation. Damage classification, vehicle make/model recognition and insurance pricing are future work.

Abstract

This project develops a vision-based car-part detection and segmentation application using a fine-tuned YOLOv8n-seg network. The model was trained on the Carparts-Seg secondary dataset, containing 3,833 images and 20,374 polygon annotations spanning 23 classes. A two-stage application combines a generic COCO-pretrained YOLOv8n detector for whole-vehicle bounding boxes with the project-specific segmentation network for individual vehicle parts. Predictions are exposed through a Python command-line interface, a FastAPI service and an HTML/CSS/JavaScript/Bootstrap user interface. Training was conducted on a Google Colab NVIDIA Tesla T4 for 25 epochs following a two-epoch smoke test. On the provided test split, preliminary box mAP@0.50 was 0.7082 and mask mAP@0.50 was 0.7110. The public frontend is deployed to Vercel; the model-backed API has been validated locally but is not publicly hosted. The work demonstrates a reproducible academic baseline for part segmentation while explicitly documenting unresolved near-duplicate risk, limited class coverage and dataset licensing uncertainty.

1. Introduction and Problem Statement

Vehicle inspection workflows often require identifying the location of visible car components. Manual part labeling is repetitive, and conventional image-level classification cannot describe the position or shape of each component. This project investigates instance segmentation: identifying each detected part with a class label, confidence score, bounding box and pixel-level polygon mask.

The immediate research problem is **car-part segmentation**, not damage assessment. The proposed system supports visual inspection and provides a foundation for later damage-to-part association; no damage type or monetary estimate is produced in the current release.

2. Project Objectives and Scope

• Fine-tune a compact YOLOv8 segmentation model for the dataset's 23 original part classes.

• Preserve model weights and class mappings for reproducible inference.

• Provide annotated image outputs, confidence scores and structured JSON results.

• Expose the model through a FastAPI backend and a browser-based frontend.

• Publish code, trained weights, evaluation documentation and a public project demonstration.

• Document technical limitations, licensing constraints and preliminary evaluation status.

Out of scope

Damage detection (dent, scratch, crack, broken part and paint damage), car make/model/year classification, and insurance or repair price estimation are **not implemented**. The frontend displays these as future-scope features, not measured capabilities.

3. Dataset Collection and Description

The project uses the secondary **Carparts-Seg** dataset distributed by Ultralytics. Its curated archive includes 3,833 640 x 640 images and YOLO-format segmentation-polygon labels. The dataset source is linked rather than re-hosted in the public repository.

Dataset property	Verified value
Total images	3,833

Dataset property	Verified value
Training / validation / test	3,156 / 401 / 276
Polygon annotations	20,374
Number of classes	23
Image resolution	640 x 640
Annotation format	YOLO segmentation polygons
Source	Ultralytics Carparts-Seg documentation and official archive

3.1 Original class labels

0: back_bumper | 1: back_door | 2: back_glass | 3: back_left_door

4: back_left_light | 5: back_light | 6: back_right_door | 7: back_right_light

8: front_bumper | 9: front_door | 10: front_glass | 11: front_left_door

12: front_left_light | 13: front_light | 14: front_right_door | 15: front_right_light

16: hood | 17: left_mirror | 18: object | 19: right_mirror

20: tailgate | 21: trunk | 22: wheel

3.2 Quality and governance checks

The project audit identified 135 empty label files (116 train, 12 validation, 7 test), treated as missing annotation rather than confidently labeled background. These were quarantined in the Colab training copy without modifying the source archive. A corrected class/split YAML was retained in `training/carparts-seg.yaml`. The dataset has class imbalance; the catch-all `object` class has very limited examples and should not be considered reliable.

The original Roboflow project states CC BY 4.0, while the curated 3,833-image distribution does not establish an explicit dataset license. Consequently, commercial rights are **not confirmed**; dataset images are not redistributed in the public GitHub repository. The repository contains the official dataset link and a separate CC0 example image.

4. Model Architecture and Methodology

4.1 Two-model inference pipeline

Stage A — vehicle detection: generic pretrained `yolov8n.pt` predicts whole-car bounding boxes. This model is not the fine-tuned project model.
Stage B — part segmentation: the fine-tuned `yolov8n-seg` weights at `models/part_detection/best.pt` predict part class, confidence, bounding box and segmentation mask. Part segmentation can succeed even when generic car detection returns zero cars.

Input	Processing	Output
Vehicle JPG/PNG/WEBP	Generic car detector + fine-tuned part segmentation	Car boxes, part masks, part labels and scores
Predictions	Image annotation and JSON serialization	Annotated image + machine-readable JSON
Frontend upload	FastAPI POST /predict (when backend available)	Prediction panel and annotated result

4.2 Software architecture

The inference implementation is in `predict.py` and `api/main.py`. The API provides `GET /health` for model availability and integrity checks, and `POST /predict` for uploaded images. The response includes car detections, part detections, confidence values, bounding boxes, mask-present flags and an annotated image encoded as base64. The frontend uses plain HTML, CSS, JavaScript and Bootstrap.

The model runs locally on CPU for inference. The frontend is publicly deployed on Vercel, but the FastAPI backend has **not** been publicly deployed; therefore the public page presents measured sample predictions and project information rather than a working cloud inference endpoint.

5. Training Procedure

A two-epoch smoke test was used to verify the training pipeline. Its weights were discarded. The released project weights were generated by a separate full fine-tuning run initialized from `yolov8n-seg.pt`.

Training parameter	Configuration
Network	YOLOv8n-seg (fine-tuned)
Compute	Google Colab NVIDIA Tesla T4 (~15 GB VRAM)
Training epochs	25 (full run)
Smoke test	2 epochs, discarded
Input image size	640
Batch size	16
Patience	10
Saved artifact	models/part_detection/best.pt
Weight size	7,045,940 bytes
Weight MD5	c99271ab36b92d655ea03dc5e1f4cb3a

The Colab training notebook and generated training summary are included in the repository under `training/`. Exact learning-rate schedules and augmentation details should be read from that notebook; they are not independently restated here.

6. Evaluation and Experimental Results

All results in this section are PRELIMINARY. They were measured using the shipped dataset splits. An unresolved cross-split perceptual-hash review identified 191 candidate near-duplicate pairs (28 at Hamming distance 6 or below; 163 at distances 7–12). These candidates have not been reviewed image by image, and a leakage-free test set has not yet been certified. Accordingly, reported metrics may be optimistic.

Split	Precision	Recall	Box mAP50	Box mAP50-95	Mask mAP50
Validation (401)	0.6016	0.7921	0.6820	0.5699	0.6973
Test (276)	0.6285	0.8188	0.7082	0.5948	0.7110

Mask mAP50-95 was not reported in the supplied training summary and is therefore not claimed. The numbers above are detection/segmentation metrics, not general image classification accuracy.

6.1 Per-class box mAP50

ID	Class	Test mAP50 (box)
0	back_bumper	0.7973
1	back_door	0.8246
2	back_glass	0.7949
3	back_left_door	0.6522
4	back_left_light	0.3536
5	back_light	0.6554
6	back_right_door	0.4024
7	back_right_light	0.3785
8	front_bumper	0.8751
9	front_door	0.8397
10	front_glass	0.9154
11	front_left_door	0.6173
12	front_left_light	0.5016
13	front_light	0.7203
14	front_right_door	0.3671
15	front_right_light	0.3362
16	hood	0.8950
17	left_mirror	0.4061
18	object	0.0000
19	right_mirror	0.3572
20	tailgate	0.8584
21	trunk	0.7300
22	wheel	0.4009

Strong test performance was measured for front_glass (0.9154), hood (0.8950) and front_bumper (0.8751). Smaller or ambiguous classes generally performed worse; object scored 0.0000 and is unreliable. Tailgate has a small sample

count, making its score sensitive to test composition.

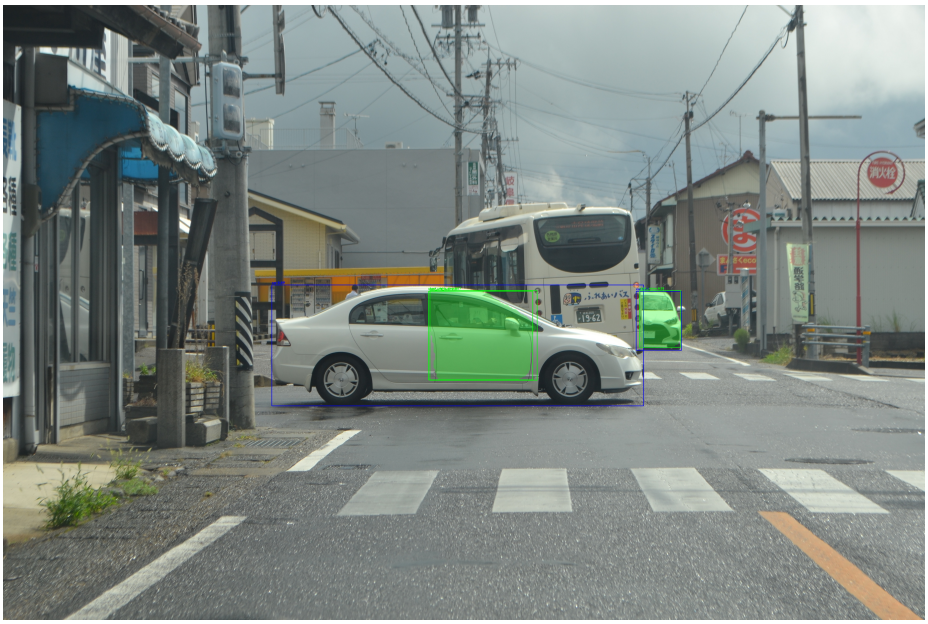
7. Inference Validation and Sample Results

The package includes a CC0 Honda Civic sample photo and a corresponding annotated prediction. The local standalone predictor returned two generic whole-car detections (confidence 0.8856 and 0.5680) and three low-confidence segmented parts: right_mirror 0.2864, back_left_door 0.2854, and back_right_door 0.2653. The low part confidences are reported as measured; this is an out-of-distribution example, not a benchmark score. The API response was also checked against the CLI output.

Figure 1. Sample input image (CC0).



Figure 2. Annotated output generated by the trained part model and generic car detector.



Three additional test-split prediction records are documented in `training/EVALUATION.md` and `training/sample_predictions_colab.json`. Their corresponding dataset images are intentionally not republished due to licensing uncertainty.

8. Application, Reproducibility and Deployment

8.1 Public repository and weights

Repository: github.com/rafaqatali-dev/AI-Car-Parts-Detection-YOLOv8. The trained weights are committed at `models/part_detection/best.pt`; the separate root-level `yolov8n.pt` is only the generic whole-car detector. The repository contains dependencies, class mapping, dataset source documentation, notebook, evaluation summary, example assets, CLI and API.

8.2 Reproduction commands

```
pip install -r requirements.txt
python predict.py --image examples/sample_civic_cc0.jpg --output-dir outputs/predict_test
uvicorn api.main:app --host 127.0.0.1 --port 8000
python -m http.server 8080 --directory frontend
```

The GUI then opens locally at `http://127.0.0.1:8080/index.html`, with API base URL `http://127.0.0.1:8000`. The automated evaluator should verify the weight checksum and use the CLI to test its own image rather than assume that the public Vercel frontend hosts a GPU/CPU inference backend.

8.3 Public frontend

Public site: https://ai-car-parts-detection-yolov8-front.vercel.app/. It displays model scope, measured example predictions, preliminary metrics, dataset description and GitHub link. The API URL is not configured for public inference, so a visitor cannot currently submit an image for remote model inference. This limitation is disclosed on the site.

9. Limitations, Ethical Considerations and Future Work

• Cross-split near-duplicate candidates remain unresolved; final leakage-free performance is not established.

• Class coverage is limited to the original 23 dataset labels and does not include every automotive component.

• Some classes are rare or ambiguous, especially object, mirrors and small lights.

• Commercial reuse of the curated dataset is not cleared; source images are not republished.

• Public deployment currently includes the static frontend only; live model inference requires separate backend hosting.

• No damage classification, car make/model/year recognition or repair/insurance price estimation is implemented.

Future work includes a frozen, deduplicated test set; improved class balance and representative Pakistani vehicle photos with appropriate consent; a separately trained damage model; damage-to-part association; and production-ready backend deployment. Any commercial transition requires explicit dataset rights and operational security review.

10. Conclusion

This semester project delivers a fine-tuned YOLOv8n-seg car-part segmentation model, preserved weights, an inference CLI, a tested local FastAPI API, a working local GUI and a public Vercel presentation website. The model produces part-level masks and confidence scores and achieved preliminary test box mAP50 of 0.7082 and mask mAP50 of 0.7110 on the shipped test split. The project is suitable as a documented academic prototype, not yet as a validated commercial damage-insurance system.

References and Project Links

Ultralytics, Carparts-Seg dataset documentation: https://docs.ultralytics.com/datasets/segment/carparts-seg

Ultralytics official dataset archive: https://github.com/ultralytics/assets/releases/download/v0.0.0/carparts-seg.zip

Ultralytics YOLO documentation: https://docs.ultralytics.com/

Gianmarco Russo, original car-seg project (Roboflow), 2023; see attribution in repository dataset notes.

Project source, notebook and metrics: https://github.com/rafaqatali-dev/AI-Car-Parts-Detection-YOLOv8

Public demonstration frontend: https://ai-car-parts-detection-yolov8-front.vercel.app/

Evidence basis: the accompanying GitHub submission ZIP, especially training/TRAINING_SUMMARY.md, training/EVALUATION.md, datasets/README.md, README.md, and the included sample image and prediction artifacts. No unreported experiment or unsupported accuracy claim has been added.